CSS Styling, Layout & Responsive Design Lab

Restyle the Lab 1 HTML portfolio using an external CSS architecture

Duration: 3 Hours | Assessment: Individual | Total Marks: 100 | **No Inline Styles Allowed**

Overview

In this lab you will take the HTML-only portfolio from Lab 1 and progressively enhance it with a full CSS architecture. You will strip all inline style="" attributes, then rebuild the presentation layer using an external stylesheet with custom properties, the Box Model, Flexbox, CSS Grid, and media queries. By the end, your portfolio must look polished on mobile (320px), tablet (768px), and desktop (1200px+).

Learning Outcomes

By the end of this lab, you will be able to:

- Write an external CSS file linked from multiple HTML pages using a shared design token system
- Apply the CSS Box Model precisely — distinguishing margin, padding, and border on real components
- Build horizontal navigation and two-column layouts using Flexbox properties
- Construct responsive card grids and sidebar layouts using CSS Grid
- Write mobile-first media queries that adapt layouts across three breakpoints
- Style form elements and apply pseudo-classes and pseudo-elements for interaction feedback

CRITICAL RULE: No inline style="" attributes may remain in any HTML file after this lab. All visual styling must live exclusively in style.css. Submissions found to contain inline styles will lose marks for each affected task.

Task 1 CSS Architecture & Custom Properties

Set up an external stylesheet and define a design token system in :root

15 marks

Objective

Create a single style.css file linked from every HTML page. Define a design token system using CSS custom properties (variables) in :root — this becomes the single source of truth for all colours, fonts, and spacing across the entire site.

Requirements

- Remove every style="" attribute from all six HTML files — grep your code; none may remain
- Link `style.css` from the `<head>` of every page using `<link rel="stylesheet" href="style.css">`

- Define at least 12 custom properties in :root covering: 2 primary colours, 2 accent colours, 3 neutral greys, 2 font families, and 3 spacing values (--space-sm, --space-md, --space-lg)
- Import at least one Google Font using `@import` at the very top of `style.css` and reference it via a custom property (e.g. `--font-heading: 'Playfair Display', serif`)
- Write a CSS reset block: at minimum box-sizing: border-box, remove default margin/padding on body, and set a base font-size of 62.5% on html (makes 1rem = 10px for easy calculation)
- Apply the :root variables throughout the stylesheet — no hard-coded hex values or bare px font sizes anywhere else

Example :root Block

```
:root {
  --color-primary:    #1a1a2e;
  --color-secondary:  #16213e;
  --color-accent:     #e8d5b7;
  --color-text:       #1a1a1a;
  --color-muted:      #666666;
  --color-surface:    #f5f4f0;
  --font-body:        'Lora', Georgia, serif;
  --font-heading:     'Playfair Display', serif;
  --space-sm:         0.5rem;
  --space-md:         1.5rem;
  --space-lg:         3rem;
}
```

CSS Properties Covered

`:root`	`var()`	`@import`	`link rel`	`box-sizing`	`font-size: 62.5%`

Task 2 Typography, Colour & the Box Model

Style all text, colour surfaces, and control spacing with margin/padding/border

15 marks

Objective

Style every text element consistently using your custom properties. Demonstrate explicit, intentional control of the CSS Box Model on at least five distinct component types — show you understand what margin, padding, and border each do.

Requirements

- Set a typographic scale using rem units: h1–h4, p, small, blockquote, code, and caption must all have explicit font-size, line-height, and color
- Style the <header>, <nav>, <main>, and <footer> landmarks with distinct background-color, padding, and color — all values via CSS variables
- Demonstrate margin vs padding on five components — add a CSS comment on each explaining which you used and why
- Style the <a> tag in all four states: :link, :visited, :hover, :active — each must be visually distinct
- Apply border-radius to at least three component types (buttons, cards, nav links)

- Add box-shadow to at least one card or panel for depth

Hint: Use outline: none on inputs carefully — always replace it with an accessible alternative focus indicator such as box-shadow: 0 0 0 3px var(--color-accent) so keyboard users can see where focus is.

CSS Properties Covered

rem / em	line-height	margin / padding	border	:hover :focus	box-shadow

Task 3 Navigation & Header with Flexbox

Build a sticky horizontal nav and a two-column hero header using Flexbox

15 marks

Objective

Use Flexbox to build the site's navigation bar and the page header. Both components must adapt gracefully at all screen sizes before any media queries are introduced.

Requirements

- Style the <nav> as a horizontal Flexbox container: site name/logo on the left, link list on the right — use justify-content: space-between
- Apply position: sticky; top: 0; to the nav with an appropriate z-index so it stays visible during scroll
- Use flex-wrap: wrap and gap on the nav link list so links wrap naturally on narrow screens without needing a media query
- Style the header as a Flexbox row: text block left, profile image placeholder right — use align-items: center
- Apply a CSS transition on nav links: transition: color 0.2s, background-color 0.2s — hover must change both
- Add an .active class to the correct <a> tag on each page and style it distinctly (a coloured bottom border works well)

Note: Add class="active" manually to the correct <a> in the nav on each HTML page. This hands-on task reinforces the connection between HTML class attributes and CSS class selectors — a fundamental relationship in front-end development.

CSS Properties Covered

display: flex	justify-content	align-items	flex-wrap	gap	position: sticky	z-index	transition

Task 4 Page Layout with CSS Grid

20

Implement a card grid and a sidebar layout using CSS Grid marks

Objective

Apply CSS Grid to create two different layouts on two different pages. Grid must be used for both a responsive multi-column card arrangement and a main-content + sidebar two-column page structure.

Requirements

- On `index.html`: style the page-cards section as a CSS Grid using `grid-template-columns: repeat(auto-fit, minmax(260px, 1fr))` — cards fill the row and wrap automatically
- On `skills.html` or `about.html`: create a two-column layout with a wider main content area and a narrower aside sidebar using `grid-template-columns: 1fr 280px`
- Use gap between all grid items — at least 1.5rem
- Use `grid-column: span 2` to make at least one element span the full width of a grid (e.g. a featured card or a section heading)
- Style each card with equal height using align-content or grid-template-rows so cards in the same row match visually
- Add a :hover transform (`transform: translateY(-4px)`) on cards with `transition: transform 0.2s` for a 'lift' effect

Hint: auto-fit vs auto-fill — with auto-fit, empty tracks collapse and items stretch to fill the row. With auto-fill, empty tracks remain, leaving gaps at the end of a row. Use auto-fit for card grids where you want items to always span the full container width.

CSS Properties Covered

`display: grid`	`grid-template-columns`	`repeat()`	`minmax()`	`auto-fit`	`gap`	`grid-column: span`	`translateY`

Task 5 Responsive Design with Media Queries

Make the portfolio fully responsive across mobile, tablet, and desktop

20 marks

Objective

Apply a mobile-first CSS strategy. Write base styles for small screens first, then use @media rules to progressively enhance the layout for wider viewports. The portfolio must function correctly at 320px, 768px, and 1280px.

Requirements

- Adopt a mobile-first approach: base styles target 320px and up; enhancements at min-width: 600px (tablet) and min-width: 1024px (desktop)
- At mobile: nav links collapse to a vertical stack; header becomes a single column; all Grid layouts revert to a single column using grid-template-columns: 1fr

- At tablet (600px+): the card grid switches to 2 columns; the header becomes a two-column Flex row
- At desktop (1024px+): full 3-column card grid; sidebar layout activates; nav is fully horizontal
- Add <meta name="viewport" content="width=device-width, initial-scale=1.0"> to every HTML page if not already present
- Make images responsive: max-width: 100%; height: auto; on all <img> elements
- Document the three breakpoints in a comment block at the very top of style.css

Required Breakpoint Structure in style.css

```
/* ============================================
   BREAKPOINTS
   mobile-first base: 320px and up (default)
   tablet:   @media (min-width: 600px)
   desktop:  @media (min-width: 1024px)
   ============================================ */

/* --- Base styles (mobile) --- */
.card-grid {
  display: grid;
  grid-template-columns: 1fr;
  gap: var(--space-md);
}

@media (min-width: 600px) {
  .card-grid { grid-template-columns: repeat(2, 1fr); }
}

@media (min-width: 1024px) {
  .card-grid { grid-template-columns: repeat(3, 1fr); }
}
```

Hint: Mobile-first means your default CSS is for small screens. Media queries then add complexity for larger screens. The alternative — writing desktop styles first and overriding for mobile — is harder to maintain and often produces heavier CSS that mobile devices still have to parse.

CSS Properties Covered

`@media`	`min-width`	`mobile-first`	`max-width: 100%`	`flex-direction: column`	`viewport meta`

Task 6 Form Styling & Visual Polish

Style the contact form and apply pseudo-classes, pseudo-elements, and transitions

15 marks

Objective

Style the contact form into a professional, accessible interface. Then apply finishing touches across the whole site using CSS pseudo-classes, pseudo-elements, and transitions to elevate the design beyond basic layout.

Requirements

- Style all form inputs, textareas, and selects consistently: uniform padding, border, border-radius, and font-family — they must not look like browser defaults
- Add a visible :focus state to every interactive form element using box-shadow in your accent colour — do not rely on browser default outlines alone
- Style the submit button: background-color transition on :hover and transform: scale(0.97) on :active
- Use the `::before` or `::after` pseudo-element on at least two elements — e.g. a decorative quote mark on the blockquote (`content: '"'`), or a coloured left bar on section headings
- Use `:nth-child` or `:nth-of-type` to apply alternating row colours to the academic results table (e.g. `tbody tr:nth-child(even)`)
- Use `:first-child` and `:last-child` to remove the top border of the first and bottom border of the last item in at least one list

CSS Properties Covered

`::before` / `::after`	`:focus`	`:nth-child`	`:first-child`	`:last-child`	`transition`	`transform: scale`	`content: ''`

Deliverables

Submit your entire portfolio folder as a .zip file. It must contain all of the following:

`style.css` Single external stylesheet — all tasks	**`index.html`** Home with card Grid layout	**`about.html`** Typography & sidebar layout
`skills.html` Lists with Flexbox styling	**`academic.html`** Table with :nth-child rows	**`projects.html`** Responsive image grid
`contact.html` Fully styled form		

Submission Checklist

- ☐ Zero style="" attributes remain in any HTML file
- ☐ All pages share one style.css — no per-page stylesheets
- ☐ Site renders correctly at 320px, 768px, and 1280px (test in browser DevTools)
- ☐ All interactive elements have a visible :focus style
- ☐ style.css uses only variables defined in :root — no loose hex values
- ☐ Breakpoints are documented in a comment block at the top of style.css
- ☐ A .active class is applied to the current page's nav link on every HTML file
- ☐ Submit as a .zip folder via the course portal before end of the lab session

Marks Summary

#	Task	Marks	Score
1	CSS Architecture & Custom Properties	15	
2	Typography, Colour & the Box Model	15	
3	Navigation & Header with Flexbox	15	
4	Page Layout with CSS Grid	20	
5	Responsive Design with Media Queries	20	
6	Form Styling & Visual Polish	15	
	TOTAL	**100**	

Good luck!